Section II: Database Management Systems-II

Sr. No.	Q2) 1) a)	Q2) 1) a)	OR Question
1	Function	Trigger	Procedure
2	Function	Function	Procedure
3	Cursor	Trigger	Procedure
4	Cursor	Trigger	Procedure
5	Cursor	Trigger	Procedure
6	Function	Function	Procedure
7	Cursor	Trigger	Procedure
8	Function	Trigger	Procedure
9	Trigger	Function	Procedure
10	Function	Trigger	Procedure
11	Cursor	Trigger	Procedure
12	Trigger	Function	Procedure
13	Trigger	Function	Procedure
14	Trigger	Function	Procedure
15	Cursor	Trigger	Procedure
16	Trigger	Cursor	Procedure
17	Trigger	Cursor	Procedure
18	Cursor	Trigger	Procedure
19	Function	Trigger	Procedure
20	Function	Trigger	Procedure
21	Function	Trigger	Procedure
22	Function	Trigger	Procedure
23	Cursor	Trigger	Procedure
24	Function	Trigger	Procedure
25	Cursor	Trigger	Procedure
26	Function	Trigger	Procedure
27	Cursor	Trigger	Procedure
28	Function	Trigger	Procedure
29	Cursor	Trigger	Procedure
30	Function	Trigger	Procedure

Page 1, 12, 14, 19, 22:
Consider the following Entities and their Relationships for Project-Employee database.
Project (pno integer, pname char (30), ptype char (20), duration integer)
Employee (eno integer, ename char (20), qualification char (15), joining_date date)
The Relationship between Project and Employee is many to many with descriptive attribute start_date date, no_of_hours_worked integer.
Constraints: Primary Key, duration should be greater than zero, pname should not be null.
Queries for creating database:

1) create table project (pno int primary key, pname char (30) not null, ptype char (20), duration integer check (duration > 0));
2) create table employee (eno integer primary key, ename char (20), qualification char (15), joining_date date);
3) create table project_employee (pno integer references project, eno integer references employee, start_date date, noofhours integer);

Q2) 1) a): Write a stored function to accept eno as input parameter and count number of projects on which that employee is working.

Function:

```
create or replace function count_projects (arg_eno integer) returns integer as $$
declare
        project_count integer;
begin
        select count(distinct pno) into project_count
        from project_employee
        where eno = arg_eno;

        return project_count;
end;
$$ language plpgsql;
```

Q2) 1) b): Write a trigger before inserting into a project table to check duration should be always greater than zero. Display appropriate message

Function:

```
create or replace function check_duration()
returns trigger as $$
begin
        if new.duration <= 0 then
                raise exception 'duration must be greater than zero.';
        end if;
        return new;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger duration_check_trigger
before insert on project
for each row
execute function check_duration();
```

Q2) 1) c)Write a stored function to accept project name as input and print the names of employees working on the project. Raise an exception for an invalid project name.

Q.2) 2) Write a procedure to display addition, subtraction and multiplication of three numbers.

Procedure:

```
create or replace procedure calc_operations( in a integer, b integer, in c integer) as $$
begin
        raise notice 'addition = %', a + b + c;
        raise notice 'subtraction = %', a - b - c;
        raise notice 'multiplication = %', a * b * c;
end;
$$ language plpgsql;
```

Q2) 2) A) Write a procedure to count odd numbers from given range (m to n) (for).

Q2) 2) B) Write a procedure to display a number in word (Using Case) and loop.

Q2) 2) C) Write a procedure to find minimum and maximum from two numbers.

Q2) 2) D) Write a procedure to search the given number is in given range.

Page 2, 15, 29:
Consider the following Entities and their Relationships
Person (pno integer, pname varchar (20), birthdate date, income money)
Area (aname varchar (20), area_type varchar (5))
An area can have one or more persons living in it, but a person belongs to exactly one area.
Constraints: Primary Key,area_type can be either 'urban' or 'rural'.

Queries for creating database:

1) create table area (ano int primary key, aname varchar (20), area_type varchar (5) check (atype in ('Urban', 'Rural')));
2) create table person (pno integer primary key, pname varchar (20), birthdate date, income money, ano int references area);

Q2) 1) a) Write a stored function to print total number of persons of a particular area. Accept area name as input parameter.

Function:

```
create or replace function count_persons (arg_aname varchar(20)) returns integer as $$
declare
        pcount integer;
begin
        select count(distinct p.ano) into pcount
        from area a, person p
        where a.ano=p.ano and aname = arg_aname;
        return pcount;
end;
$$ language plpgsql;
```

Q2) 1) b) Write a stored function to print sum of income of person living in "------" area type. (Accept area type as input parameter). Display appropriate message for invalid area type.

Function:

```
create or replace function sum_income (arg_atype varchar(20)) returns integer as $$
declare
        t_income integer;
begin
        select sum(income) into t_income
        from area a, person p
        where a.ano=p.ano and atype = arg_atype;
        if t_income is NULL then
            raise exception 'Invalid area type';
        end if;
        return t_income;
end;
$$ language plpgsql;
```

Q2) 1) c) Write a cursor to update the income of all people living in 'Urban' area by 10%.
Q2) 1) d) Write a trigger before insert the record of person if the pnumber is negative give the message "Invalid Number".
Q2) 1) e) Write a cursor to display the names of persons living in urban area.
Q2) 1) f) Write a trigger before deleting a person's record from the person's table. Raise a notice and display the message "person record is being deleted".
Q2) 2) Write a procedure to display division of two numbers. Use raise to display error messages for division by zero error.

Procedure:

```
create or replace procedure divide_numbers(num1 integer, num2 integer) as $$
Declare
        result float;
begin
        if num2 = 0 then
```

```
                raise exception 'error: division by zero is not allowed.';
            end if;
            result := num1 / num2;
            raise notice 'result = %', result;
        end;
        $$ language plpgsql;
```

Q2) 2) B) Write a procedure to display all odd numbers from 1 to 100.

Q2) 2) C) Write a procedure to display subtraction of three numbers.

Page 3:
Consider the following Entities and their Relationship.
Student (s_no integer, s_name char (20), address char (25), class char (10))
Teacher (t_no integer,t_name char (10), qualification char (10),experience integer)
Relationship between Student and Teacher is Many to Many with descriptive attribute subject and marks_scored.
Constraints: Primary Key,s_name,t_name should not be null,marks_scored> 0

Queries for creating database:

1) create table student (sno integer primary key, sname char(20) not null, address char(25), class char(10));
2) create table teacher (tno integer primary key, tname char(10) not null, qualification char(10), experience integer);
3) create table student_teacher (sno integer references student, tno integer references teacher, subject varchar(10), marks_scored integer check (marks_scored > 0));

Q1) a) Write a cursor which will accept student number from the user and display student name along with teacher name who taught 'RDBMS' subject.

Function:

```
create or replace function get_student_teacher_rdbms(arg_sno int) returns void as $$
declare
        rec record;

        cur_stud cursor for select s.sname, t.tname
                            from student s, teacher t, student_teacher st
                            where s.sno = st.sno and st.tno = t.tno and st.subject = 'rdbms'
                                and s.sno = arg_sno;
begin
    open cur_stud;

    loop
        fetch cur_ stud into rec;
        exit when not found;

        raise notice 'student name: %, teacher name: %', rec.sname, rec.tname;
    end loop;

    close cur_ stud;
end;
$$ language plpgsql;
```

Q1) b) Write a trigger before insert the record of student. If student number is less than or equal to zero give message "Invalid Number".

Function:

```
create or replace function check_student_number()
returns trigger as $$
begin
    if new.sno <= 0 then
            raise exception 'invalid number';
    end if;
    return new;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger trg_check_student_number
before insert on student
for each row
execute function check_student_number();
```

Q2) Write a procedure to find minimum and maximum from two numbers.

Procedure:

```
create or replace procedure find_min_max(num1 int, num2 int, out min_val int, out max_val int )
language plpgsql as $$
begin
   if num1 < num2 then
        min_val := num1;
        max_val := num2;
   else
        min_val := num2;
        max_val := num1;
   end if;
end;
$$;
```

Page 4:
Consider the following Entities and their Relationships .
Movies (m_name varchar (25), release_year integer, budget money)
Actor (a_name char (30), role char (30), charges money, a_address varchar(30))
Producer (producer_id integer, name char (30), p_address varchar (30))
The relationships are as follows: Each actor has acted in one or more movies. Each producer has produced many movies and each movie can be produced by more than one producers. Each movie has one or more actors acting in it, in different roles.

Queries for creating database:

1) create table movies (mno int primary key, mname varchar (25), releaseyear integer, budget money);
2) create table actor (ano int primary key, aname char (30), role char (30), charges money, aaddress varchar(30));
3) create table producer (pid integer primary key, pname char (30), paddress varchar (30));
4) create table movie_actor(mno int references movies, ano int references actor);
5) create table movie_producer(mno int references movies, pno int references producer);

Q2) 1) a) Write a cursor to pass actor name as input parameter and return total number of movies in which given actor is acting.

Function:

```
create or replace function display_count_m (arg_aname varchar (20)) returns integer
language plpgsql as $$
declare
	movie_count integer := 0;
	t_ano integer;

	cur_movies cursor for select ma.mno from movie_actor ma where ma.ano = actor_id;

	rec record;
begin
	select ano into t_ano from actor where aname = arg_aname;
	if t_ano is null then
		raise exception 'actor not found.';
	end if;

	open cur_movies;

	loop
		fetch cur_movies into rec;
		exit when not found;
		movie_count := movie_count + 1;
	end loop;

	close cur_movies;

	return movie_count;
end;
$$;
```

Q2) 1) b) Write a trigger before inserting into movie table to check budget. Budget should be minimum 50 lakhs. Display appropriate message.

Function:

```
create or replace function check_movie_budget() returns trigger
language plpgsql as $$
begin
	if new.budget < 5000000 then
		raise notice 'Error: budget should be at least 50 lakhs.';
		return null;
	end if;
	return new;
end;
$$;
```

Trigger:

```
	create trigger trg_check_budget
	before insert on movies
	for each row
	execute function check_movie_budget();
```

Q2) 2) Write a procedure to check the number is positive, negative or zero.

Procedure:

```
create or replace procedure check_number(num numeric)
language plpgsql as $$
begin
	if num > 0 then
		raise notice 'number is positive';
	elsif num < 0 then
		raise notice 'number is negative';
	else
		raise notice 'number is zero';
	end if;
end;
$$;
```

Page 5:
Consider the following Entities and their Relationships for Student-Competition
Student (sreg_no int ,s_name varchar(20), s_class char(10))
Competition (c_no int ,c_name varchar(20), c_type char(10))
Relationship between Student and Competition is many to many with descriptive attributes rank and year. Constraints: Primary Key, c_type should not be null, c_type can be 'sport' or 'academic'.

Queries for creating database:

1) Create table student (sreg_no int primary no, s_name varchar(20), s_class char(10));
2) Create table competition (c_no int primary key, c_name varchar(20), c_type char(10) not null check (c_type in ('sport', 'academic'));
3) Create table student_competition (sreg_no int references student, c_no int references competition, rank int, year int);

Q2) 1) a) write a function using cursor to accept year and class to display student name, competition name and rank.

Function:

```
create or replace function student_competition_details(arg_year int, arg_class varchar(10))
returns void language plpgsql as $$
declare
rec record;
cur_details cursor for select s.s_name, c.c_name, sc.rank
                        from student s, student_competition sc, competition c
                         where s.sreg_no = sc.sreg_no and c.c_no = sc.c_no and sc.year = arg_year
                                and s.s_class = arg_class;
begin
        open cur_details;

        loop
                fetch cur_details into rec;
                exit when not found;

                raise notice 'student: %, competition: %, rank: %', rec.s_name, rec.c_name, rec.rank;
        end loop;

        close cur_details;
end;
$$;
```

Q2) 1) b) Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

Function:

```
create or replace function before_update_competition() returns trigger
language plpgsql as $$
begin
	raise notice 'competition record is being updated';
	return new;
end;
$$;
```

Trigger:

```
create trigger trg_before_update_competition
before update on competition
for each row
execute function before_update_competition();
```

Q2) 2) Write a procedure to find maximum and minimum from three numbers.

Procedure:

```
create or replace procedure find_max_min(a int, b int, c int)
language plpgsql as $$
declare
	max_val int;
	min_val int;
begin
	max_val :=  a;
	if b>max_val then
		max_val := b;
	end if;
	if c>max_val then
		max_val := c;
	end if;
	min_val :=  b;
	if b<min_val then
		min_val := b;
	end if;
	if c>min_val then
		min_val := c;
	end if;

	raise notice 'maximum = %, minimum = %', max_val, min_val;
end;
$$;
```

Page 6, 13:
Consider the following Entities and their Relationships
BUS (bus_no int , capacity int , depot_name varchar(20))
ROUTE (route_no int, source char(20), destination char(20),no_of_stations int)
DRIVER (driver_no int , driver_name char(20), license_no int, address char(20), d_ageint , salary float)
The relationships are as: BUS_ROUTE: M-1 BUS_DRIVER: M-M with descriptive attributes Date of duty allotted and Shift (it can be 1 (Morning) or 2 (Evening)). Constraints: 1. License_no is unique. 2. Bus capacity is not null

Queries for creating database:

1) Create table route (route_no int primary key, source char(20), destination char(20),no_of_stations int);
2) Create table bus (bus_no int primary key, capacity int not null, depot_name varchar(20), route_no int references route);
3) Create table driver (driver_no int primary key, driver_name char(20), license_no int unique, address char(20), d_age int , salary float);
4) Create table bus_driver(bus_no int references bus, driver_no int references driver, duty_date date, shift int check (shift in (1,2)));

Q2) 1) a) Write a stored function to accept the bus number and print driver name allotted to that bus.

Function:

```
create or replace function get_driver_by_bus(arg_bus_no int) returns void
language plpgsql as $$
declare
	rec record;

	found_status boolean default false;
begin
	for rec in select d.driver_name
			from driver d, bus_driver bd
			where d.driver_no = bd.driver_no and bd.bus_no = arg_bus_no
	loop
		raise notice 'driver name: %', rec.driver_name;
		found_status := true;
	end loop;
	if not found_status then
		raise notice 'no driver assigned to this bus.';
	end if;
end;
$$;
```

Q2) 1) b) Write a stored function to accept depot name and display driver details having age more than 50.

Function:

```
create or replace function get_driver_by_bus(arg_depot_name varchar(20)) returns void
language plpgsql as $$
declare
	rec record;
	found_status boolean default false;
begin
	for rec in select d. driver_no, driver_name, license_no, address, d_age, salary
			from driver d, bus_driver bd
		where d.driver_no = bd.driver_no and depot_name = arg_depot_name and d_age > 50
	loop
		raise notice 'driver name: %', rec.driver_name;
		found_status := true;
	end loop;
	if not found_status then
		raise notice 'no driver assigned to this bus.';
	end if;
end;
$$;
```

Q2) 1) a) Define a trigger before insert the record of driver if the age is not between 18 and 50, raise an error message “invalid entry”.

Q2) 1) b) Write a stored function to accept the bus_no and date and print its allotted drivers. Raise an exception in case of invalid bus number.

Q2) 2) Write a procedure to find number is even or odd.

Procedure:

```
create or replace procedure check_even_odd(num int)
language plpgsql as $$
begin
	if num % 2 = 0 then
		raise notice 'number is even';
	else
		raise notice 'number is odd';
	end if;
end;
$$;
```

Q2) 2) A) Write a procedure to search the given number is in given range.

Page 7:
Consider the following Entities and their Relationships
Branch (br_id integer, br_name char (30), br_city char (10))
Customer (cno integer, c_name char (20), caddr char (35), city char (20))
Loan_application(lno integer, l_amt_required money, l_amt_approved money, l_date date)
Relationship between Branch, Customer and Loan_application is Ternary.
Ternary (br_id integer, cno integer, lno integer)
Constraints: Primary Key, l_amt_required should be greater than zero.

Queries for creating database:

1) Create table branch (brid integer primary key, brname char (30), brcity char (10));
2) Create table customer (cno integer primary key, cname char (20), caddr char (35), city char (20));
3) Create table loan_application(lno integer primary key, lamt_required money check (lamt_required >0), lamt_approved money, ldate date);
4) Create table Ternary (brid integer references branch, cno integer references customer, lno integer references loan_application);

Q2) 1) a) Write a stored function using cursor to count the number of customers of particular branch. (Accept branch name as input parameter).

Function:

```
create or replace function count_customers_by_branch(arg_brname varchar) returns integer as $$
declare
      rec record;
      total_count integer := 0;
      cur_customer cursor for select c.cno
                                from branch b, customer c, ternary t
                                where b.brid = t.brid and t.cno = c.cno and b.brname = arg_brname;
begin
    open cur_customer;

    loop
          fetch cur_customer into rec;
          exit when not found;
          total_count := total_count + 1;
    end loop;

    close cur_customer;

    if total_count = 0 then
          raise exception 'invalid branch name or no customers found';
    end if;

    return total_count;
end;
$$ language plpgsql;
```

Q2) 1) b) Write a trigger before insert record of customer. If the customer number is less than or equal to zero, then give the appropriate message.

Function:

```
create or replace function check_customer_number()
returns trigger as $$
begin
	if new.cno <= 0 then
		raise exception 'invalid customer number';
	end if;
	return new;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger trg_check_customer_number
before insert on customer
for each row
execute function check_customer_number();
```

Q2) 2) Write a procedure to find sum of first 20 numbers (using unconditional loop).

Procedure:

```
create or replace procedure sum_first_20()
language plpgsql as $$
declare
	i int := 1;
	total int := 0;
begin
	loop
		total := total + i;
		i := i + 1;
		exit when i > 20;
	end loop;
	raise notice 'sum of first 20 numbers is: %', total;
end;
$$;
```

Consider the following Entities and their Relationships
TRAIN: (train_no int, train_name varchar(20), depart_time time , arrival_time time, source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int)
PASSENGER : (passenger_id int, passenger_name varchar(20), address varchar(30), age int ,gender char)
Relationships: Train _Passenger: M-M relationship named ticket with descriptive attributes as follows TICKET: (train_no int, passenger_id int, ticket_no int ,bogie_no int, no_of_berths int ,tdate date , ticket_amt decimal(7,2),status char)
Constraints:The status of a berth can be 'W' (waiting) or 'C' (confirmed).

Queries for creating database:

1) Create table train (trno int primary key, trname varchar(20), depart_time time , arrival_time time, source_stn varchar (20),dest_stn varchar (20), no_of_res_bogies int ,bogie_capacity int) ;
2) Create table passenger (pid int primary key, pname varchar(20), address varchar(30), age int ,gender char);
3) Create table ticket (trno int references train, pid int references passenger, ticket_no int ,bogie_no int, no_of_berths int ,tdate date , ticket_amt decimal(7,2),status char check(status in (‘W’, ‘C’)));

Q2) 1) a) Write a stored function to display the ticket details of a train. (Accept train name as input parameter). Raise an exception in case of invalid train name.

Function:

```
create or replace function get_ticket_details(arg_trname varchar) returns void as $$
declare
    rec record;
    found_flag boolean := false;
begin
    for rec in select t.ticket_no, p.pname, t.bogie_no, t.no_of_berths, t.tdate, t.ticket_amt, t.status
                from train tr, ticket t, passenger p
                where tr.trno = t.trno and t.pid = p.pid and  tr.trname = arg_trname
    loop
        found_flag := true;
        raise notice 'ticket no: %, passenger: %, bogie: %, berths: %, date: %, amount: %, status: %',
        rec.ticket_no, rec.pname, rec.bogie_no, rec.no_of_berths, rec.tdate, rec.ticket_amt, rec.status;
    end loop;

    if not found_flag then
        raise exception 'invalid train name or no tickets found';
    end if;
end;
$$ language plpgsql;
```

Q2) 1) b) Write a trigger after insert on passenger to display message “Age above 5 will be charged full fare” if age of passenger is more than 5.

Function:

```
create or replace function check_passenger_age() returns trigger as $$
begin
	if new.age > 5 then
		raise notice 'age above 5 will be charged full fare';
	end if;
	return new;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger trg_check_passenger_age
after insert on passenger
for each row
execute function check_passenger_age();
```

Q2) 2) Write a procedure to display addition of two numbers.

Procedure:

```
create or replace procedure add_two_numbers(num1 int, num2 int)
language plpgsql as $$
declare
	result int;
begin
	result := num1 + num2;
	raise notice 'addition is: %', result;
end;
$$;
```

Page 9, 17:
Consider the following database maintained by a school about students and competitions.
STUDENT (sreg_no int , name char(30), class char(10))
COMPETITION (c_no int , name char(20), C_type char(15))
The relationship is as follows: STUDENT-COMPETITION: M-M with described attributes rank, year and prize(int).

Queries for creating database:

1) Create table student (sreg_no int primary key, name char(30), class char(10));
2) Create table competition (c_no int primary key, name char(20), C_type char(15));
3) Create table student_competition (sreg_no int references student, c_no int references competition, rank int, year int, prize int);

Q2) 1) a) Define a trigger before updating a competition table. Raise a notice and display message "competition record is being updated".

Function:

```
create or replace function msg_competition_update() returns trigger as $$
begin
   raise notice 'competition record is being updated';
   return new;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger trg_msg_competition_update
before update on competition
for each row
execute function msg_competition_update();
```

Q2) 1) b) Write a function using cursor which accepts a name of student and returns the total no of prizes won by that student in the year 2020.

Function:

```
create or replace function count_prizes_2020(arg_name varchar)
returns integer as $$
declare
        rec record;
        total_prizes integer := 0;
        cur_prizes cursor for select sc.prize from student s, student_competition sc
        where s.sreg_no = sc.sreg_no and s.name = arg_name and sc.year = 2020;
begin
        open cur_prizes;
        loop
           fetch cur_prizes into rec;
           exit when not found;
           total_prizes := total_prizes + rec.prize;
        end loop;
        close cur_prizes;
        if total_prizes = 0 then
              raise exception 'invalid student name or no prizes in year 2020';
        end if;
        return total_prizes;
end;
$$ language plpgsql;
```

Q2) 2) Write a procedure to display division of two numbers use raise to display divided by zero exception messages.

Procedure:

```
create or replace procedure divide_numbers(num1 integer, num2 integer) as $$
Declare
        result float;
begin
        if num2 = 0 then
                raise exception 'error: division by zero is not allowed.';
        end if;
        result := num1 / num2;
        raise notice 'result = %', result;
end;
$$ language plpgsql;
```

Q2) 2) Write a procedure to find sum of first 50 numbers (using unconditional loop).

Consider the following Entities and their Relationships
Book(b_no int, b_name varchar (20), pub_name varchar (10), b_price float)
Author (a_no int, a_name varchar (20), qualification varchar (15), address varchar (15))
Relationship between Book and Author is many to many.
Constraints: Primary Key,pub_name should not be null.

Queries for creating database:

1) Create table book(b_no int primary key, b_name varchar (20), pub_name varchar (10) not null, b_price float);
2) Create table author (a_no int primary key, a_name varchar (20), qualification varchar (15), address varchar (15));
3) Create table book_author(b_no int refernces book, a_no int references author);

Q2) 1) a) Write a stored function to display the book details written by author. (Accept Author name as input parameter). Raise an exception in case of invalid author name.

Function:

```
create or replace function get_books_by_author(arg_aname varchar) returns table (b_no int,
 b_name varchar, pub_name varchar, b_price float )  as $$
declare
    author_count int;
begin
      select count(*) into author_count
      from author
      where a_name = arg_aname;
      if author_count = 0 then
                raise exception 'invalid author name';
      end if;

      return query
      select b.b_no, b.b_name, b.pub_name, b.b_price
      from book b, book_author ba, author a
      where b.b_no = ba.b_no and a.a_no = ba.a_no and a.a_name = arg_aname;
end;
$$ language plpgsql;
```

Q2) 1) b) Write a trigger after insert on book to display message "prize is so high" if book price is more than 1000.

Function:

```
create or replace function check_book_price() returns trigger as $$
begin
      if new.b_price > 1000 then
            raise notice 'price is so high';
      end if;
      return new;
end;
$$ language plpgsql;
```

Trigger:

```
      create trigger trg_check_price
      after insert on book
      for each row
      execute function check_book_price();
```

Q2) 2) Write a procedure to display all even numbers from 1 to 50.

Procedure:

```
create or replace procedure display_even_numbers()
language plpgsql as $$
declare
	i int default 2;
begin
	while i <= 50 loop
		raise notice '%', i;
		i := i + 2;
	end loop;
end;
$$;
```

Consider the following Entities and their Relationships
Student (rollno, s_name , class)
Subject (scode , subject_name)
Relationship between Student and Subject is many to many with descriptive attribute marks.

Queries for creating database:

1) Create table student (rollno int primary key, s_name varchar(20), class varchar(15));
2) Create table subject (scode varchar(10), subject_name varchar(20));
3) Create table student_subject (rollno int references student, scode varchar(10) references subject, marks integer);

Q2) 1) a) Write a stored function using cursor to calculate total marks of each student and display it.

Function:

```
create or replace function total_marks_all_students() returns void as $$
declare
    rec record;
    total integer;
cur_student cursor for select rollno, s_name from student;
begin
    open cur_student;
    loop
        fetch cur_student into rec;
        exit when not found;
        select sum(marks) into total from student_subject where rollno = rec.rollno;
        raise notice 'roll no: %, name: %, total marks: %', rec.rollno, rec.s_name, total;
    end loop;
    close cur_student;
end;
$$ language plpgsql;
```

Q2) 1) b) Define a trigger before deleting a student record from student table. Raise a notice and display message "student record is being deleted".

Function:

```
create or replace function before_delete_student() returns trigger
as $$
begin
    raise notice 'student record is being deleted';
    return old;
end;
$$ language plpgsql;
```

Trigger:

```
create trigger trg_before_delete_student
before delete on student
for each row
execute function before_delete_student();
```

Q2) 2) Write a procedure to find sum and average of first n numbers using conditional loop(while)

Procedure:

```
create or replace procedure sum_avg_n(n int)
language plpgsql as $$
declare
	i int := 1;
	total int := 0;
	avg_val float;
begin
	while i <= n loop
		total := total + i;
		i := i + 1;
	end loop;
	avg_val := total::numeric / n;
	raise notice 'sum = %, average = %', total, avg_val;
end;
$$;
```

Consider the following Student –Subject database [15 Marks] Student(rollno integer, name varchar(30), address varchar(50), class varchar(10)) Subject(scode varchar(10), subject_name varchar(20)) Student-Subject are related with M-M relationship with attributes marks_scored. Q2) Using above Database Solve the following [10 Marks] 1) Define a trigger before deleting a student record from the student table. Raise a notice and display the message "student record is being deleted". OR 1) Write a stored function using cursors, to calculate total marks of each student and display it. 2) Write a procedure to check the number is positive and negative.

Page 18

Consider the following Item-Supplier database Item(itemno integer, itemname varchar(20)) Supplier(supplier_no integer, supplier_name varchar(20), city varchar(20)) The relationship is as, [15 Marks] Item-Supplier: M-M relationship with rate(money) and quantity (integer) as descriptive attributes. Q2) Using above Database Solve the following 1. Write a cursor to display the names of items whose rate is more than 500. OR [10 Marks] 1. Write a trigger before insert or update on rate field. If the rate is less than 50 then raise the appropriate exception. 2. Write a procedure to find the maximum number from three numbers.

Page 20

Consider the following Department-employee database. Department (dno integer, dname varchar(20),city varchar(20)) Employee (eno integer, ename varchar(20), salary money) Department and Employee are related with a one to many relationships. Q2) Using above Database Solve the following [15 Marks] [10 Marks] 1) Write a function to accept department name and display the maximum salary of an employee in that department. OR 1) Write a trigger before insert/update on an employee record. Raise exception if salary if salary
2) Write a procedure to display multiplication of two numbers.

Page 21

Consider the following database [15 Marks] Doctor(d_no int, d_name varchar(30), specialization varchar(35), phone_no int, charges int) Hospital(h_no int, h_name varchar(20), city varchar(10)) Doctor and Hospital are related with many to one relationship. Q2) Using above Database Solve the following [10 Marks] 1) Write a function which will accept the name of the hospital and calculate the average charges of doctors visiting that hospital. OR 1) Write a trigger before insert/update on Doctor. Raise exception if charges are <0. 2) Write a procedure to find sum and average of first n numbers using conditional loop(while).

Page 23
Consider the following database [15 Marks] Car (c_no int, owner varchar(20), model varchr(10), color varchar(10) Driver (driver_no int, driver_name varchar(20), license_no int, d_age int, salary float) Car and Driver are related with many to many relationship Q2) Using above Database Solve the following [10 Marks] 1) Write a cursor which accepts the driver name and prints the details of all cars that this driver has driven, if the driver name is invalid, print an appropriate message. OR 1) Write a trigger before insert/update on Driver. Raise exception if driver age is < 21. 2) Write a procedure to find sum of first 100 numbers (using unconditional loop).

Page 24
Consider the following Department-employee database. Department (dno integer, dname varchar(20),city varchar(20)) Employee (eno integer, ename varchar(20), salary money) Department and Employee are related with a one to many relationship Q2) Using above Database Solve the following [15 Marks] [10 Marks] 1) Write a function to accept department name as input and display employee name along with salary of that department. OR 1) Write a trigger after insert on an employee record. Display appropriate message when the record is inserted. 2) Write a stored procedure to display first 20 even numbers.

Page 25
Consider the following database Customer (cno integer, cname varchar(20), city varchar(20)) Account (a_no int, a_type varchar(10), opening_date date, balance money) Customer and Account are related with one to many relationship Q2) Using above Database Solve the following [15 Marks] [10 Marks] 1. Write a function using cursor which accepts city name as input and prints the details of all customers in that city. OR 1. Write a trigger which does not allow deletion of accounts of type. 2.Write a procedure to display all customers from 'Pune' city.

Page 26
Consider the following database : Item (itemno integer, Itemname varchar(20), quantity integer) Supplier(supplierno integer ,Supplier name varchar(20),city varchar(20)) [15 Marks] Item and supplier are related with many to many relationship. Rate is descriptive attribute. Q2) Using above Database Solve the following [10 Marks] 1) Write a stored function using cursors, to accept Item name from the user and display the Rate and Supplier Name for that Item. OR 2) Write a trigger before update on rate field. If the difference in the old rate and new rate is more than Rs 2000, raise an exception and display the corresponding message. 3) Write a procedure to display subtraction of three numbers.

Page 27
Consider the following database : Student (rollno integer, name varchar(30),class varchar(10)) Subject(Scode varchar(10),subject name varchar(20)) [15 Marks] Student and subject are related with M-M relationship with attributes marks_scored. Q2) Using above Database Solve the following [10 Marks] 1. Write a stored function using cursors, to accept class from the user and display the details of the students of that class. OR 1. Write a trigger before insert/update the marks_scored. Raise exception if Marks are negative. 2. Write a procedure to display all even numbers from 1 to 50.

Page 28
Consider the following database : Company (Name varchar(30),address (50),city varchar(20), share_value money) Person (pname varchar(30),pcity varchar (20)) Company and Person are related with M to M relationship with descriptive attribute No_of_shares Q2) Using above Database Solve the following [10 Marks] 1) Write a stored function to update the share_values by 20% for Person " ". OR 1) Write a trigger before deleting company record. Display

appropriate message to the user. 2) Write a procedure to display division of two numbers use raise to display error messages.

Page 30

Consider the following database Student (Roll_No int, Sname varchar (20), Sclass char (10)) Teacher (T_No int, Tname char (20), Experience int) [15 Marks] Student and Teacher are related with many to many relationship with the descriptive attribute Subject. Q2) Using above Database Solve the following [10 Marks] 1) Write a stored function to count the number of teachers having experience > 10 years OR 1) Write a trigger before insert the record of the student in the Student table. If the Roll_No is less than or equal to zero then the trigger gets fired and displays the message "Invalid Roll Number". 2) Write a procedure to find sum and average of first n numbers using conditional loop(while)